КОМП’ЮТЕРНИЙ ПРАКТИКУМ №4

« Вивчення криптосистеми RSA та алгоритму електронного
підпису; ознайомлення з методами генерації параметрів для
асиметричних криптосистем
»

Виконали: Ромашко Мирослава та

Чупай Яна, група ФБ-41
Перевірила: Селюх Поліна Валентинівна

Мета та основні завдання роботи:
Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів
для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту
інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

ХІД РОБОТИ

*1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або
заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на
простоту. В якості датчика випадкових чисел використовуйте вбудований генератор
псевдовипадкових чисел вашої мови програмування. В якості тесту перевірки на простоту рекомендовано використовувати тест Міллера-Рабіна із попередніми пробними діленнями. Тести необхідно реалізовувати власноруч, використання готових реалізацій тестів не дозволяється. Доповнення до п. 1. Для одного зі згенерованих простих чисел навести проміжні результати перевірки тестом Міллера-Рабіна для однієї з використаних основ.*

Пробні ділення. Число спершу перевіряється на подільність на 168 простих до 1000. Це швидко відкидає більшість складених кандидатів.

Тест Міллера-Рабіна. Записуємо $p - 1 = 2^s \cdot d$, де d непарне. Для випадкової основи x рахуємо послідовність x^d, x^(2d), x^(4d), … за модулем p. Якщо p просте, то ця послідовність або одразу починається з 1 чи −1, або на якомусь кроці дає −1 перед тим, як дійти до 1. Складене число зазвичай порушує це правило для більшості основ. Ми використала 20 основ, тож імовірність помилки не перевищує $4^{-20} \approx 10^{-12}$. У перевірці умови врахований і випадок $x^d \equiv -1$ ($r = 0$), як у визначенні сильно псевдопростого числа з теорії.

Генерація. Береться випадкове x з інтервалу [2^255, 2^256 − 1] (рівно 256 біт), далі перебираються непарні числа x, x+2, … до першого, що пройшло обидва тести.

```python
# Допоміжні функції

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a


def power_mod(x, e, m):
    y = 1
    for bit in bin(e)[2:]:
        y = y * y % m
        if bit == "1":
            y = y * x % m
    return y


def small_primes(limit):
    sieve = [True] * (limit + 1)
    sieve[0] = sieve[1] = False
    for i in range(2, int(limit ** 0.5) + 1):
        if sieve[i]:
            for j in range(i * i, limit + 1, i):
                sieve[j] = False
    return [i for i, is_p in enumerate(sieve) if is_p]


SMALL_PRIMES = small_primes(1000)     # 168 малих простих чисел для пробних ділень
```

```python
# Тест Міллера-Рабіна

def decompose(p):
    d, s = p - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    return s, d

```

Допоміжні функції

Функція *gcd(a, b)* обчислює найбільший спільний дільник двох чисел за алгоритмом Евкліда: доки b не нуль, пара (a, b) замінюється на (b, a mod b). У тесті вона перевіряє, що вибрана основа x взаємно проста з p.

Функція *power_mod(x, e, m)* піднесення до степеня за модулем за схемою Горнера. Показник e розглядається в двійковому вигляді, і біти проходяться від старшого до молодшого: на кожному кроці проміжний результат підноситься до квадрата за модулем m, а якщо біт дорівнює 1, ще множиться на x за модулем m. Так замість e множень виконується лише близько $\log_2 e$ піднесень до квадрата й множень, і всі числа залишаються меншими за m.

Функція *small_primes(limit)* будує за решетом Ератосфена список усіх простих чисел до limit. Для пробних ділень використовується список SMALL_PRIMES із 168 простих чисел до 1000.

Функція *decompose(p)* подає p − 1 у вигляді $2^s \cdot d$ з непарним d, ділячи p − 1 на 2, доки воно парне. Результатом є пара (s, d), потрібна для тесту Міллера-Рабіна.

```python

# Пробні ділення

def trial_division(n):
    for q in SMALL_PRIMES:
        if n == q:
            return None
        if n % q == 0:
            return q
    return None

```

```python

def strong_pseudoprime(p, x, trace=None):
    s, d = decompose(p)
    xr = power_mod(x, d, p)
    if trace is not None:
        trace.append((0, xr))
    if xr == 1 or xr == p - 1:
        return True
    for r in range(1, s):
        xr = xr * xr % p
        if trace is not None:
            trace.append((r, xr))
        if xr == p - 1:
            return True
        if xr == 1:
            return False
    return False


def miller_rabin(p, rounds=20):
    for _ in range(rounds):
        x = random.randint(2, p - 2)
        if gcd(x, p) != 1:
            return False, x
        if not strong_pseudoprime(p, x):
            return False, x
    return True, None
```

```python
def is_prime(n, rounds=20):
    if n < 2:
        return False
    if n <= SMALL_PRIMES[-1]:
        return n in SMALL_PRIMES
    if trial_division(n) is not None:
        return False
    return miller_rabin(n, rounds)[0]

# Генерація випадкового простого числа

def random_prime(bits, rounds=20, log=None):
    n0, n1 = 2 ** (bits - 1), 2 ** bits - 1
    if log is not None:
        log.update({"start": None, "tested": 0, "trial": [], "mr": []})
    while True:
        x = random.randint(n0, n1)
        m = x if x % 2 == 1 else x + 1
        if log is not None and log["start"] is None:
            log["start"] = x
        while m <= n1:
            if log is not None:
                log["tested"] += 1
            q = trial_division(m)
            if q is not None:
                if log is not None:
                    log["trial"].append((m, q))
            else:
                ok, witness = miller_rabin(m, rounds)
                if ok:
                    return m
                if log is not None:
                    log["mr"].append((m, witness))
            m += 2
```

Основні функції

Функція *trial_division(n)* реалізує тест пробних ділень: число n послідовно ділиться на малі прості числа, і якщо воно ділиться на одне з них (і не дорівнює йому), повертається цей дільник, а число визнається складеним. Інакше повертається None. Цей тест швидко відкидає більшість складених чисел перед ресурсомістким тестом Міллера-Рабіна.

Функція *strong_pseudoprime(p, x)* перевіряє, чи є p сильно псевдопростим за основою x. Вона обчислює x^d mod p, і якщо результат дорівнює 1 або p − 1 (тобто −1), число проходить перевірку. Інакше результат послідовно підноситься до квадрата за модулем p не більше s − 1 разів: якщо отримано −1, число проходить перевірку, а якщо отримано 1, число складене, бо 1 з'явилась, минувши −1. Якщо жоден з варіантів не настав, p не є сильно псевдопростим за основою x. Необов'язковий параметр trace дозволяє зберегти проміжні значення для звіту.

Функція *miller_rabin(p, rounds)* реалізує сам тест Міллера-Рабіна. Вона вибирає rounds випадкових основ x з інтервалу від 2 до p − 2, для кожної перевіряє gcd(x, p) = 1 і сильну псевдопростоту. Якщо хоч за однією основою число не пройшло перевірку, воно складене, і функція повертає цю основу як свідка. Якщо пройшло за всіма, число вважається простим, а ймовірність помилки не перевищує 4^(−rounds), тобто для 20 основ близько 10^{-12}.

Функція *is_prime(n, rounds)* об'єднує обидва тести: спочатку пробні ділення, потім Міллер-Рабін.

Функція *random_prime(bits, rounds, log)* генерує випадкове просте число довжиною рівно bits біт. Вона випадково вибирає x з інтервалу [2^(bits−1), 2^bits − 1], переходить до найближчого непарного числа і перебирає непарні числа x, x+2, x+4, … Кожне з них перевіряється спочатку пробними діленнями, потім тестом Міллера-Рабіна, і перше число, що пройшло обидва, повертається як результат.

Якщо в межах інтервалу простого не знайдено, процедура повторюється з новим випадковим x. Необов'язковий словник log збирає хід пошуку: кількість кандидатів і числа, відхилені кожним з тестів.

```
Генерація випадкового простого числа довжиною 256 біт
Початкове випадкове число x:
  879695762670319432725813400668267666646581409704940306942512670595277202553 97
Перевірено непарних кандидатів: 167
  відхилено пробними діленнями:     144
  відхилено тестом Міллера–Рабіна:  22

Перші 3 кандидати, відхилені пробними діленнями (число, дільник):
  879695762670319432725813400668267666646581409704940306942512670595277202553 97
    ділиться на 3
  879695762670319432725813400668267666646581409704940306942512670595277202554 01
    ділиться на 13
  879695762670319432725813400668267666646581409704940306942512670595277202554 03
    ділиться на 3

Кандидати, відхилені тестом Міллера–Рабіна (число, основа-свідок):
  879695762670319432725813400668267666646581409704940306942512670595277202553 99
    основа x = 553932228821322470005083175515069274113090664219556682118501177544559772695 88
  879695762670319432725813400668267666646581409704940306942512670595277202554 31
    основа x = 354927120564228935562649144044882465894574876015183000331134307145091608204 0
  879695762670319432725813400668267666646581409704940306942512670595277202554 59
    основа x = 556936546910705059084972650606883724610260179779804031921508843809079494878 19
  879695762670319432725813400668267666646581409704940306942512670595277202554 83
    основа x = 453078625753029896535909001320555676541373973318020191770107469267103239472 5
  879695762670319432725813400668267666646581409704940306942512670595277202554 97
    основа x = 602786975345902074642356883933281981190857685398869104271012310083530318707 64
  879695762670319432725813400668267666646581409704940306942512670595277202555 01
    основа x = 965451623960534998796201170486633301317696157035006749670156469948281209885
  879695762670319432725813400668267666646581409704940306942512670595277202555 07
    основа x = 747240885637292686738294897050222798597667145044888607776243436609473024654 03
  879695762670319432725813400668267666646581409704940306942512670595277202555 33
    основа x = 730019314340340890544680792998771298156302852999654005583386280178981794579 13
  879695762670319432725813400668267666646581409704940306942512670595277202555 37
    основа x = 451120280397140326047836005950830409360868975470329727440590426625005389259 15
```

```
Знайдене просте число p (256 біт):
  879695762670319432725813400668267666646581409704940306942512670595277202557 29

Проміжні результати тесту Міллера–Рабіна для p:
  p − 1 = 2^4 * d, де d = 549809851668949645453633375417667291654113381065587691839070419122048251 5983
  основа x = 839905550279970453151827460228318930710039857262455473025680748806867284080, gcd(x, p) = 1
  r = 0: x^(2^0 * d) mod p = 867134929823127408570248509426441174091483491269383113198159479233362942082 36
  r = 1: x^(2^1 * d) mod p = 752417498950479643391946817627587120535130817143430426932661041569447962875 65
  r = 2: x^(2^2 * d) mod p = 396553359812011828797171011669242183782788389909496030974614043250673916405 66
  r = 3: x^(2^3 * d) mod p = 879695762670319432725813400668267666646581409704940306942512670595277202557 28  (= −1 mod p)
  p сильно псевдопросте за основою x: True
```

Програма вибрала випадкове число x довжиною 256 біт і перевірила 167 непарних кандидатів, починаючи з x. 144 кандидати (86%) відхилено пробними діленнями, оскільки вони діляться на малі прості числа, наприклад на 3 або 13. 22 кандидати пройшли пробні ділення, але були відхилені тестом Міллера-Рабіна: для кожного знайдено основу-свідка, за якою число не є сильно псевдопростим, а отже воно складене. 167-й кандидат пройшов обидва тести, і це шукане просте число p. Така кількість кандидатів нормальна, бо густина простих чисел поблизу N становить близько 1/ln N, для 256-бітних чисел ln N ≈ 177, тож серед непарних простим є приблизно кожне 89-те. Частка кандидатів без дільників менших за 1000 (23 із 167, тобто 14%) також збігається з теоретичною, близько 16%. Проміжні результати перевірки: $p - 1 = 2^4 \cdot d$, а gcd(x, p) = 1. Значення x^(2^r·d) mod p при r = 0, 1, 2 не дорівнюють ні 1, ні −1, а при r = 3 воно дорівнює $p - 1 \equiv -1 \pmod p$. Отже, p є сильно псевдопростим за цією основою. Наступне піднесення до квадрата дало б 1, що відповідає малій теоремі Ферма (x^(p−1) ≡ 1). Після

перевірки за 20 основами число вважаємо простим, імовірність помилки не перевищує $4^{-20} \approx 10^{-12}$.

2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини
щонайменше 256 біт. При цьому пари чисел беруться так, щоб $pq \le p_1q_1$; p і q - прості
числа для побудови ключів абонента A, p_1 і q_1 - абонента B.

```python
# Завдання 2: дві пари простих чисел

def generate_primes_for_two_users(bits=256, rounds=20):
    nums, logs = [], []
    while len(nums) < 4:
        log = {}
        x = random_prime(bits, rounds, log)
        if x in nums:                       # усі чотири числа мають бути різними
            continue
        log["prime"] = x
        nums.append(x)
        logs.append(log)
    pair_a, pair_b = (nums[0], nums[1]), (nums[2], nums[3])
    logs_a, logs_b = logs[0:2], logs[2:4]
    if pair_a[0] * pair_a[1] > pair_b[0] * pair_b[1]:     # A має отримати меншу пару
        pair_a, pair_b = pair_b, pair_a
        logs_a, logs_b = logs_b, logs_a
    (p, q), (p1, q1) = pair_a, pair_b
    return p, q, p1, q1, logs_a + logs_b
```

Функція *generate_primes_for_two_users* генерує чотири різні 256-бітні прості числа тією ж random_prime із завдання 1. Перша пара (p, q) призначена абоненту A, друга (p_1, q_1) абоненту B. Якщо $pq > p_1q_1$, пари міняються місцями. Умова $pq \le p_1q_1$ потрібна для протоколу розсилання ключів: A підписує k своїм секретним ключем, отримуючи $S < n$, а потім шифрує S ключем B за модулем n_1, і щоб S відновлювалося однозначно, потрібно $n \le n_1$.

Програма друкує числа, кількість відхилених кандидатів, $n = pq$ та $n_1 = p_1q_1$, а також зберігає два файли:

- primes_log.txt з повним переліком відхилених кандидатів для додатка до звіту;
- primes.json з числами для завдань 3–5, щоб у звіті скрізь були одні й ті самі p, q, p_1, q_1 (інакше кожен запуск давав би нові числа).



За допомогою функції random_prime із завдання 1 згенеровано чотири різні прості числа довжиною по 256 біт: p і q для абонента A, p_1 і q_1 для абонента B. Результати пошуку:

Число	Абонент	Кандидатів перевірено	Відхилено пробними діленнями	Відхилено тестом Міллера-Рабіна
p	A	13	12	0
q	A	24	18	5
p_1	B	25	17	7
q_1	B	20	15	4

Усього перевірено 82 кандидати: 62 (76%) відхилено пробними діленнями, 16 відхилено тестом Міллера-Рабіна, 4 пройшли обидва тести і стали шуканими числами. Кожне з чотирьох чисел є першим кандидатом у послідовності непарних чисел, що пройшов обидві перевірки. Кількість кандидатів випадкова, бо залежить від відстані від випадково вибраного початку до найближчого простого числа. У середньому для 256-бітних чисел це близько 89 непарних кандидатів, а в нашому запуску кількості менші (13–25), що є наслідком випадкового вибору, а не помилки роботи програми. Повний перелік відхилених кандидатів із їхніми малими дільниками та основами-свідками наведено в додатку (файл primes_log.txt).

Добутки p і q для кожного абонента дають модулі RSA: $n = p \cdot q$ має 512 біт, $n_1 = p_1 \cdot q_1$ також має 512 біт, і виконується умова $p \cdot q \leq p_1 \cdot q_1$ ($n < n_1$). Ця умова потрібна для протоколу розсилання ключів: абонент A підписує повідомлення k своїм секретним ключем (результат $S < n$), а потім шифрує S відкритим ключем абонента B за модулем n_1. Щоб S однозначно відновлювалося після шифрування, потрібно $S < n_1$, тобто $n \leq n_1$.

Довжина кожного з чисел 256 біт відповідає вимозі «щонайменше 256 біт». Усі чотири числа різні, а $p \neq q$ і $p_1 \neq q_1$, що необхідно для коректної роботи RSA.

3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e). За допомогою цієї функції побудувати схеми RSA для абонентів A і B - тобто, створити та зберегти для подальшого використання відкриті ключі (e, n), (e_1, n_1) та секретні d і d_1. Для однієї зі згенерованих пар ключів навести значення основних параметрів та перевірити виконання співвідношень, необхідних для коректної роботи RSA.

Додано generate_key_pair(p, q), яка повертає відкритий ключ (n, e) і секретний ключ (d, p, q). Також додано check_key_pair для перевірки співвідношень, save_keys/load_keys для збереження в keys.json і load_primes. Програма бере числа з primes.json, тож спершу має бути виконано завдання 2.

```python
def generate_key_pair(p, q, e=65537):
    n = p * q
    phi = (p - 1) * (q - 1)                # функція Ойлера phi(n)
    if gcd(e, phi) != 1:                   # e = 2^16 + 1 може не підійти; тоді беремо випадкове e
        while True:
            e = random.randrange(3, phi - 1, 2)
            if gcd(e, phi) == 1:
                break
    d = inv(e, phi)                        # e*d = 1 (mod phi(n))
    return (n, e), (d, p, q)


def check_key_pair(public, secret):
    n, e = public
    d, p, q = secret
    phi = (p - 1) * (q - 1)
    m = random.randrange(2, n - 1)         # випадкове повідомлення
    return [
        ("p і q прості (тест Міллера–Рабіна)", is_prime(p) and is_prime(q)),
        ("p != q", p != q),
        ("n = p*q", n == p * q),
        ("1 < e < phi(n)", 1 < e < phi),
        ("gcd(e, phi(n)) = 1", gcd(e, phi) == 1),
        ("0 < d < phi(n)", 0 < d < phi),
        ("e*d = 1 (mod phi(n))", e * d % phi == 1),
        ("розшифрування: (M^e)^d mod n = M", power_mod(power_mod(m, e, n), d, n) == m),
        ("підпис: (M^d)^e mod n = M", power_mod(power_mod(m, d, n), e, n) == m),
    ]
```

```python
def save_keys(path, keys):
    data = {name: {"public": {"n": str(pub[0]), "e": str(pub[1])},
                   "secret": {"d": str(sec[0]), "p": str(sec[1]), "q": str(sec[2])}}
            for name, (pub, sec) in keys.items()}
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)


def load_keys(path):
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
    return {name: ((int(v["public"]["n"]), int(v["public"]["e"])),
                   (int(v["secret"]["d"]), int(v["secret"]["p"]), int(v["secret"]["q"])))
            for name, v in data.items()}

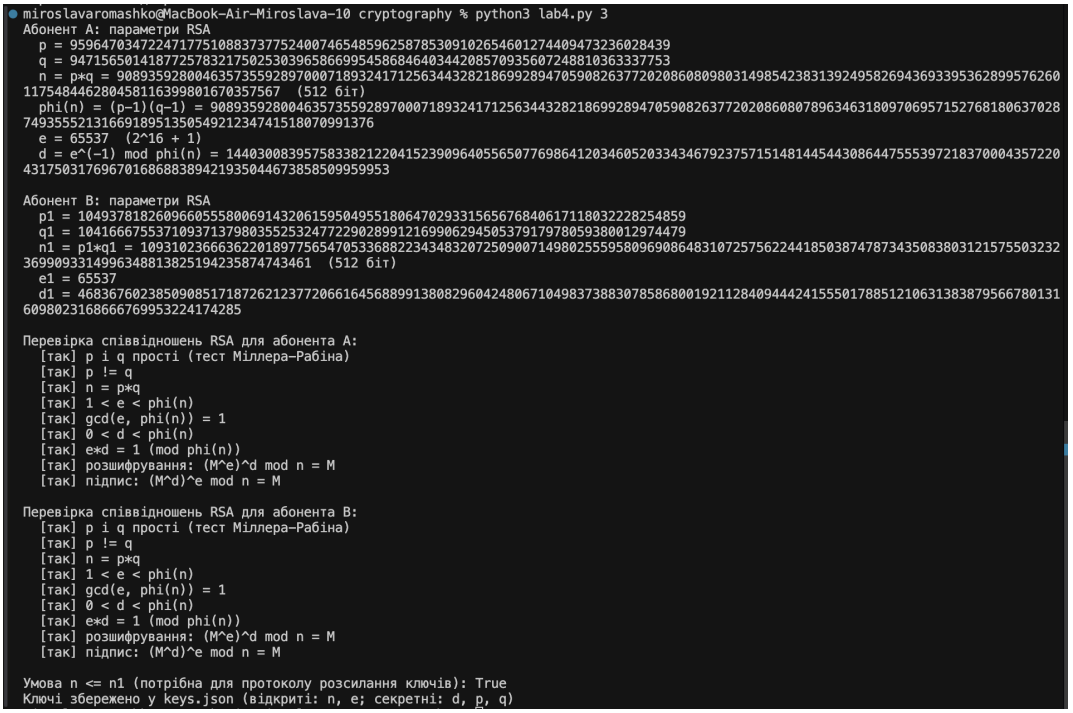

def load_primes(path="primes.json"):
    with open(path, encoding="utf-8") as f:
        d = json.load(f)
    return int(d["p"]), int(d["q"]), int(d["p1"]), int(d["q1"])

```

Для абонентів A і B побудовано схеми RSA. Для A: $n = p \cdot q$ (512 біт), $\varphi(n) = (p - 1)(q - 1)$, відкрита експонента $e = 65537 = 2^{16} + 1$ (стандартний вибір для сучасних систем, умову $\gcd(e, \varphi(n)) = 1$ перевірено; якби вона не виконувалась, програма взяла б випадкове e). Секретна експонента $d = e^{-1} \bmod \varphi(n)$ знаходиться розширеним алгоритмом Евкліда. Відкритий ключ це (n, e), секретний (d, p, q). Для B аналогічно за p_1 і q_1 отримано (n_1, e_1) і (d_1, p_1, q_1). Усі значення наведено у виводі програми і збережено у keys.json.

Для обох пар ключів перевірено співвідношення, необхідні для коректної роботи RSA:

- p і q прості та різні, $n = p \cdot q$;
- $1 < e < \varphi(n)$ і $\gcd(e, \varphi(n)) = 1$, інакше d не існує;
- $0 < d < \varphi(n)$ і $e \cdot d \equiv 1 \pmod{\varphi(n)}$, що забезпечує M^(ed) ≡ M (mod n);
- на випадковому повідомленні M: $(M^e)^d \bmod n = M$ (розшифрування) і $(M^d)^e \bmod n = M$ (перевірка підпису).



$n = p \cdot q$ і $\varphi(n) = (p - 1)(q - 1)$ збігаються з виведеними, $e \cdot d \equiv 1 \pmod{\varphi(n)}$ виконується для обох абонентів (d має 509 біт, d_1 має 511 біт), а шифрування з наступним розшифруванням повертає вихідне повідомлення для A і для B. Усі дев'ять перевірок у програмі пройшли для обох ключових пар.

4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів A і B. Кожна з операцій (шифрування, розшифрування, створення цифрового підпису, перевірка цифрового підпису) повинна бути реалізована окремою процедурою, на вхід до якої повинні подаватись лише ті ключові дані, які необхідні для її виконання.

Для кожної з чотирьох операцій створено окрему процедуру, на вхід якої подаються лише ті дані, що потрібні для її виконання. Процедура encrypt(m, e, n) шифрує повідомлення на відкритому ключі адресата за формулою C = M^e mod n і не має доступу до жодних секретних параметрів. Процедура decrypt(c, d, n) розшифровує криптограму на секретному ключі за формулою M = C^d mod n. Процедура sign(m, d, n) створює підпис S = M^d mod n, тобто «шифрує» повідомлення на секретному ключі відправника. Процедура verify(m, s, e, n) обчислює S^e mod n на відкритому ключі відправника й порівнює результат із M. Простих чисел p і q жодна з цих процедур не потребує, бо для обчислень достатньо модуля n та одного з показників e або d. Модульне піднесення до степеня виконується власною функцією power_mod методом послідовного піднесення до квадрата та множення за бітами показника. Кожна процедура

перевіряє, що її аргумент лежить у межах від 0 до n − 1, інакше викликає помилку. Крім того, verify повертає «хибно», якщо підпис S виходить за ці межі.

```python
def encrypt(m, e, n):
    """Шифрування: повідомлення та ВІДКРИТИЙ ключ адресата (e, n)."""
    if not 0 <= m <= n - 1:
        raise ValueError("повідомлення має задовольняти 0 <= M <= n-1")
    return power_mod(m, e, n)


def decrypt(c, d, n):
    """Розшифрування: криптограма та СЕКРЕТНИЙ ключ d (і модуль n)."""
    if not 0 <= c <= n - 1:
        raise ValueError("криптограма має задовольняти 0 <= C <= n-1")
    return power_mod(c, d, n)


def sign(m, d, n):
    """Створення підпису S = M^d mod n: повідомлення та СЕКРЕТНИЙ ключ d (і n)."""
    if not 0 <= m <= n - 1:
        raise ValueError("повідомлення має задовольняти 0 <= M <= n-1")
    return power_mod(m, d, n)


def verify(m, s, e, n):
    """Перевірка підпису: M, S та ВІДКРИТИЙ ключ відправника (e, n)."""
    if not 0 <= s <= n - 1:
        return False
    return power_mod(s, e, n) == m % n
```

Для обох абонентів програма випадково обрала повідомлення M у межах від 2 до n − 2, зашифрувала його, розшифрувала й порівняла результат із вихідним. Потім було створено підпис і перевірено його. Окремо перевірено, що підпис не проходить при спотворенні повідомлення (M + 1) або самого підпису (S + 1).

Абонент A (n = 9089359…70567 (154 цифри), e = 65537):

```
e = 65537
d = 1440300839575833821220415239096405565077698641203460520334346792375715148144544308644755539721837000435722043175031769670168688389421935044673
858509959953

Відкрите повідомлення M:
  2423889458391637028490094162477562239466128275311474837862558576003763756763364147213826319387135549574136491886641713521216390110154589499415189099351616
Криптограма C = M^e mod n:
  4261403489321815798520304570511476809156618133131095966311717624356911665685843650376931346347795857944646733915665901961642825901696357503890581134824046
Розшифрування M' = C^d mod n:
  2423889458391637028490094162477562239466128275311474837862558576003763756763364147213826319387135549574136491886641713521216390110154589499415189099351616
  M' == M: True
```

Абонент B (n_1 = 10931023…43461 (155 цифр), e_1 = 65537):

```
Відкрите повідомлення M:
  4813241811683581231303905786358127055768899499899564891587136672397921749612853023476605048747574131161083144173080093719321400338958885821397827411458966
Криптограма C = M^e mod n:
  2728638568133337737056905974443477552906561250589482626208980631597662408678798978749828504528957769906737781169682882427170438796267728938313576465402165
Розшифрування M' = C^d mod n:
  4813241811683581231303905786358127055768899499899564891587136672397921749612853023476605048747574131161083144173080093719321400338958885821397827411458966
  M' == M: True
```

В обох випадках розшифроване повідомлення M’ збіглося з M (M' == M: True), а підпис пройшов перевірку (True). Спотворене повідомлення M + 1 та спотворений підпис S + 1 було відхилено (False). Це підтверджує, що підпис залежить від кожного біта повідомлення, і що навіть мінімальна зміна одразу виявляється.

5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA. Протоколи роботи кожного учасника (відправника та приймаючого) повинні бути реалізовані у вигляді окремих процедур, на вхід до яких повинні подаватись лише ті ключові дані, які необхідні для виконання. Перевірити роботу програм для випадково обраного ключа 0 < k < n.

Протокол реалізовано у вигляді двох окремих процедур, які побудовано на функціях із завдання 4. Умовою його роботи є $n_1 \geq n$: підпис S лежить у межах від 0 до n − 1 і має бути зашифрований на ключі B без зведення за модулем, тому модуль абонента B не може бути меншим за модуль A. Для наших ключів ця умова виконується ($n < n_1$), про що програма виводить Умова n1 >= n: True. Якщо ж умова не виконується, send_key повідомляє про помилку, і A має згенерувати собі іншу пару ключів.

Процедура відправника send_key(k, d, n, e1, n1) приймає секрет k, секретний ключ d абонента A та відкритий ключ (e_1, n_1) абонента B. Вона обчислює підпис S = k^d mod n, а потім шифрує на ключі B і сам секрет $k_1 = k^{e_1} \bmod n_1$, і підпис $S_1 = S^{e_1} \bmod n_1$. По відкритому каналу передається лише пара (k_1, S_1). Процедура одержувача receive_key(k1, s1, d1, n1, e, n) приймає цю пару, секретний ключ d_1 абонента B та відкритий ключ (e, n) абонента A. Вона розшифровує $k = k_1^{d_1} \bmod n_1$ та $S = S_1^{d_1} \bmod n_1$, а потім перевіряє підпис рівністю k = S^e mod n. Секрет k вибрано випадково з умови 0 < k < n.

```python
def send_key(k, d, n, e1, n1):
    """
    Сторона відправника A.
    Вхід: секрет k, СЕКРЕТНИЙ ключ A (d, n), ВІДКРИТИЙ ключ B (e1, n1).
    Вихід: (k1, S1, S): S = k^d mod n, k1 = k^e1 mod n1, S1 = S^e1 mod n1.
    """
    if n1 < n:
        raise ValueError("n1 < n: абонент A має згенерувати собі іншу пару ключів")
    if not 0 < k < n:
        raise ValueError("потрібно 0 < k < n")
    s = sign(k, d, n)            # підпис на секретному ключі A
    k1 = encrypt(k, e1, n1)      # шифрування на відкритому ключі B
    s1 = encrypt(s, e1, n1)      # шифрування підпису на відкритому ключі B
    return k1, s1, s


def receive_key(k1, s1, d1, n1, e, n):
```

```
    """
    Сторона одержувача B.
    Вхід: (k1, S1), СЕКРЕТНИЙ ключ B (d1, n1), ВІДКРИТИЙ ключ A (e, n).
    Вихід: (k, S, ok): k = k1^d1 mod n1, S = S1^d1 mod n1, ok = (k == S^e mod n).
    """
    k = decrypt(k1, d1, n1)       # конфіденційність
    s = decrypt(s1, d1, n1)
    ok = verify(k, s, e, n)       # автентифікація
    return k, s, ok
```

Послідовність обчислених і переданих значень для одного виконання протоколу:

```
Секрет k (0 < k < n), який передає A:
  8851065045476509118550844753768753270573212233987530327503826794227061667019979616900370738668755270417671706837261005764819284112466232284603700620462196

--- Абонент A обчислює ---
S  = k^d mod n   (підпис на секретному ключі d A):
  2816939836077613394682723005454083895175570992058832356499754914048426564269380463899521249498350964549186048645206659334018404679811427352877061595439509
k1 = k^e1 mod n1 (шифрування на відкритому ключі e1 B):
  9981081897234783759650869974808714888874610905372328605525764629137380645713015790669398686094861489229632208999392414874064522101933183742475279253346826
S1 = S^e1 mod n1 (шифрування підпису на e1 B):
  4290812526006063451086008340403058118148612605837961095313233853689441720514712467353850616765540119952539467401523428997470468709775379531642432825443928

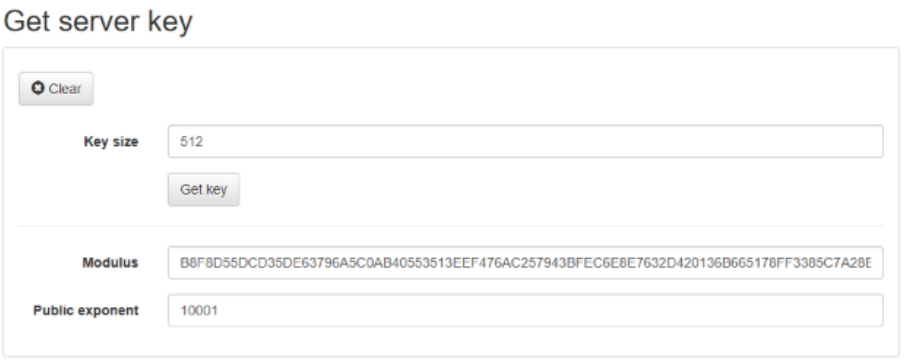
--- По відкритому каналу передається (k1, S1) ---

--- Абонент B обчислює ---
k = k1^d1 mod n1 (розшифрування на секретному ключі d1 B):
  8851065045476509118550844753768753270573212233987530327503826794227061667019979616900370738668755270417671706837261005764819284112466232284603700620462196
S = S1^d1 mod n1:
  2816939836077613394682723005454083895175570992058832356499754914048426564269380463899521249498350964549186048645206659334018404679811427352877061595439509
S^e mod n (перевірка на відкритому ключі e A):
```

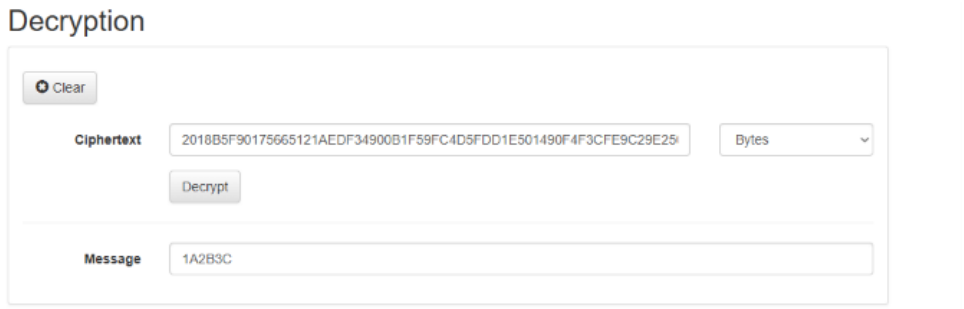
Результат: підпис вірний (k == S^e mod n: True), отриманий абонентом B секрет k збігається з надісланим (True). Спотворення значення S_1 під час передачі призводить до відхилення, бо після розшифрування виходить випадкове число, а S^e mod n ≠ k (Спотворений S1 приймається одержувачем: False).

Перевіряю операції шляхом взаємодії з тестовим середовищем http://asymcryptwebservice.appspot.com/?section=rsa

1. Шифрування: я шифрую, сервер розшифровує.

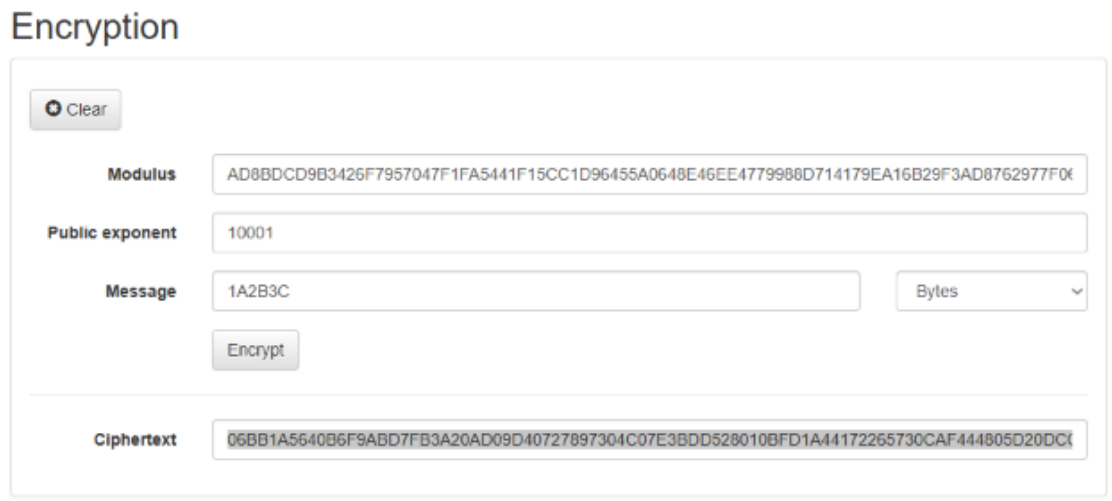



```
PS C:\Users\yana\Desktop\other_labs\crypto\lab4> python lab4.py enc --m 1A2B3C --e 10001 --n B8F8D55DCD35DE63796A5C0AB40553513EEF476AC257943BFEC6E
8E7632D420136B665178FF3385C7A28B370874C714C3ACD631B296220A4DAAB1955A39A0B09
2018B5F90175665121AEDF34900B1F59FC4D5FDD1E501490F4F3CFE9C29E250EB914C8269361E58D7BA4E5962C5A05CBB2891D188D4948F8BCE81B50D64D2440
```




```
PS C:\Users\yana\Desktop\other_labs\crypto\lab4> python lab4.py show --user A
Modulus (n), hex:
AD8BDCD9B3426F7957047F1FA5441F15CC1D96455A0648E46EE4779988D714179EA16B29F3AD8762977F065D990B92803F8ECED607E8926E7F4220EC1464D23F
Public exponent (e), hex:
10001
```

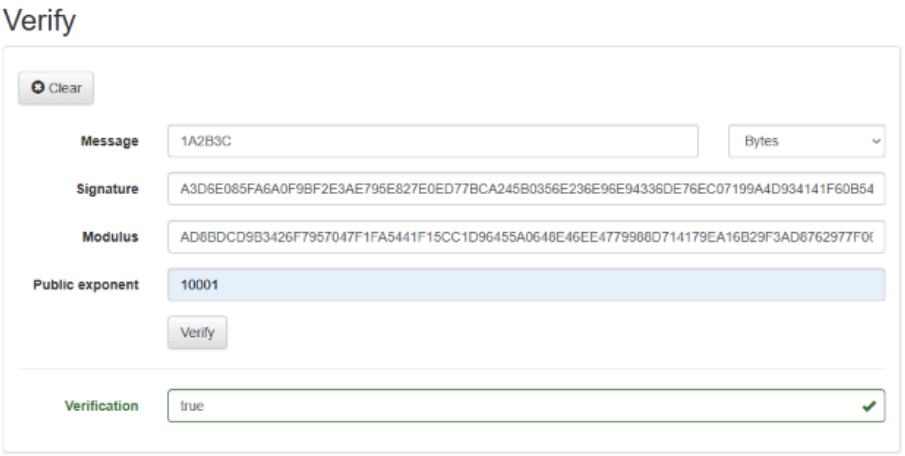
2. Шифрування: сервер шифрує, я розшифровую.



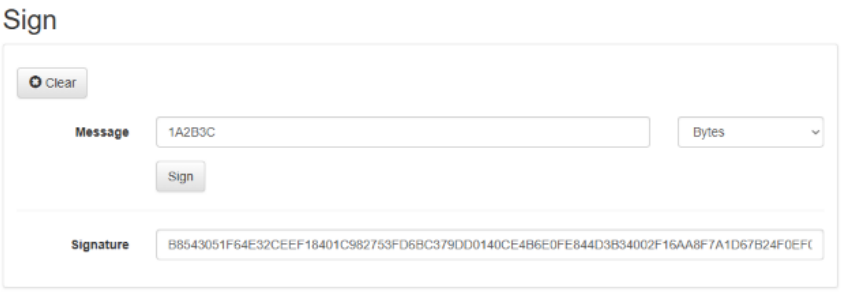

```
PS C:\Users\yana\Desktop\other_labs\crypto\lab4> python lab4.py dec --user A --c 06BB1A5640B6F9ABD7FB3A20AD09D40727897304C07E3BDD528010BFD1A4417226
5730CAF444805D20DCC669FA153F4E72F32E9FB680AB833E95E2D5F3AC8D45
1A2B3C
```

3. Підпис: я підписую, сервер перевіряє.

```
PS C:\Users\yana\Desktop\other_labs\crypto\lab4> python lab4.py sign --user A --m 1A2B3C
A3D6E085FA6A0F9BF2E3AE795E827E0ED77BCA245B0356E236E96E94336DE76EC07199A4D934141F60B5421133F2A4170F13CCE14F772DAA2C9346F663BCC68A
```



4. Підпис: сервер підписує, я перевіряю.




```
PS C:\Users\yana\Desktop\other_labs\crypto\lab4> python lab4.py verify --m 1A2B3C --s B8543051F64E32CEEF18401C982753FD6BC379DD0140CE4B6E0FE844D3B3
4002F16AA8F7A1D67B24F0EF0BD89090AE472594B3FD760A833FE0435E1C083BC0A8 --e 10001 --n B8F8D55DCD35DE63796A5C0AB40553513EEF476AC257943BFEC6E8E7632D420
136B665178FF3385C7A28B370874C714C3ACD631B296220A4DAAB1955A39A0B09
True
```

Висновки

У лабораторній роботі було реалізовано криптосистему RSA та протокол конфіденційного розсилання ключів із підтвердженням справжності відправника. Усі потрібні алгоритми написано власноруч без готових реалізацій тестів простоти: піднесення до степеня за модулем (power_mod), розширений алгоритм Евкліда, пробні ділення на 168 простих чисел до 1000 та тест Міллера-Рабіна з 20 випадковими основами.

Для пошуку випадкового простого числа довжиною 256 біт програма обирає випадкове число, доводить його до непарного й перевіряє кандидатів по черзі. У показаному запуску було перевірено 299 непарних кандидатів: 255 відхилено пробними діленнями, 43 відхилено тестом Міллера-Рабіна і 1 пройшов усі перевірки. Це наочно показує, що пробні ділення швидко прибирають більшість складених чисел, а тест Міллера-Рабіна знаходить складені числа без малих дільників. Для знайденого числа наведено проміжні результати тесту: $p - 1 = 2^1 \cdot d$, для основи x виконується $\gcd(x, p) = 1$ і $x^d \bmod p = 1$, отже p сильно псевдопросте за цією основою.

Згенеровано чотири різні прості числа довжиною 256 біт для двох абонентів так, що $p \cdot q \leq p_1 \cdot q_1$. Модулі $n = p \cdot q$ і $n_1 = p_1 \cdot q_1$ мають по 512 біт, а умова $n \leq n_1$ виконується. Для кожного абонента побудовано ключову пару з відкритою експонентою $e = 65537$ і секретною $d = e^{-1} \bmod \varphi(n)$, яку знайдено розширеним алгоритмом Евкліда. Для обох пар програма підтвердила всі співвідношення: простоту p та q, $p \neq q$, $n = p \cdot q$, $1 < e < \varphi(n)$, $\gcd(e, \varphi(n)) = 1$, $0 < d < \varphi(n)$, $e \cdot d \equiv 1 \pmod{\varphi(n)}$, а також практичну перевірку розшифрування $(M^e)^d \equiv M$ та підпису $(M^d)^e \equiv M$ на випадковому повідомленні.

Операції шифрування, розшифрування, створення та перевірки підпису реалізовано окремими процедурами, які приймають лише потрібні їм дані (модуль і один із показників). Для обох абонентів випадкове повідомлення після шифрування й розшифрування збіглося з вихідним, а створений підпис пройшов перевірку. При цьому підпис не приймається, якщо змінити повідомлення $(M + 1)$ або сам підпис $(S + 1)$, тобто підпис чутливий до будь-якого спотворення.

Протокол розсилання ключів реалізовано двома процедурами: send_key для відправника A і receive_key для одержувача B. Відправник підписує секрет k на власному секретному ключі d, а потім шифрує і k, і підпис на відкритому ключі B (за умови $n_1 \geq n$, яка гарантує, що підпис $S < n$ вміщується в модуль n_1). У результаті конфіденційність забезпечує ключова пара абонента B (шифрування на e_1, розшифрування на d_1), а підтвердження справжності відправника забезпечує ключова пара абонента A (підпис на d, перевірка на e). В експерименті B отримав значення k, що збіглося з надісланим, підпис пройшов перевірку, а при спотворенні S_1 під час передачі одержувач відхилив повідомлення.

Слід зазначити обмеження навчальної реалізації. Це «підручниковий» RSA без доповнення (padding) і без хешування: повідомлення розглядаються як числа менші за n, шифрування є детермінованим, а підпис створюється безпосередньо для M, тоді як на практиці повідомлення попередньо хешують, а шифрування виконують із випадковим доповненням. Модулі довжиною 512 біт відповідають мінімальній вимозі завдання (прості числа від 256 біт), але для реального захисту є недостатніми, і сучасні рекомендації вимагають модулів від 2048 біт. Тест Міллера-Рабіна є імовірнісним (для 20 основ ймовірність помилки не перевищує 4^{-20}), а вбудований генератор псевдовипадкових чисел Python не є криптографічно стійким. Якщо ти виконала перевірку з тестовим середовищем (шифрування та підпис у обидва боки), додай речення про те, що результати власної реалізації збіглися з результатами сервера; без цієї перевірки таке твердження робити не слід.

Контрольні питання

1. Перевірки під час генерації простих чисел.

Спочатку випадкове число доводиться до непарного (парне не може бути простим). Далі йдуть пробні ділення на 168 малих простих чисел до 1000. Вони дешево відсіюють більшість складених кандидатів (у виводі 255 з 299), бо ділення на 3, 5, 7 тощо виконується швидко. Потім застосовується тест Міллера-Рабіна з 20 випадковими основами. Спочатку перевіряється gcd(x, p) = 1, потім розкладається p − 1 = 2^s·d, обчислюється x^d mod p і послідовні квадрати. Число проходить основу, якщо x^d ≡ ±1 або знайдено x^(2^r·d) ≡ −1. Якщо хоч одна основа це спростує, число складене (43 кандидати відхилено тестом). Тест потрібен тому, що пробні ділення не знаходять великих дільників, а Міллера-Рабіна має ймовірність помилки не більше 4^{-20} для 20 основ. Додатково програма перевіряє довжину числа (256 біт) та те, що всі чотири числа різні.

2. Умови на параметри ключа RSA та їх перевірка.

Мають виконуватися: p і q прості та p ≠ q; n = p·q; 1 < e < φ(n) і gcd(e, φ(n)) = 1; 0 < d < φ(n); e·d ≡ 1 (mod φ(n)), де φ(n) = (p − 1)(q − 1); крім того, для протоколу потрібно n ≤ n_1. Функція check_key_pair перевіряє кожну умову окремо й виводить [так]/[НІ]: простоту p і q тестом Міллера-Рабіна, p ≠ q, n = p·q, 1 < e < φ(n), gcd(e, φ(n)) = 1, 0 < d < φ(n), e·d mod φ(n) = 1. Окремо виконується практична перевірка на випадковому M: (M^e)^d mod n = M (розшифрування) і (M^d)^e mod n = M (підпис). Для обох абонентів усі перевірки дали «так», а умову n ≤ n_1 перевірено окремо (True).

3. Які дані потрібні операціям.

Encrypt(M, e, n) потребує повідомлення та відкритий ключ адресата, Decrypt(C, d, n) потребує криптограму та секретний ключ d з модулем n, Sign(M, d, n) потребує повідомлення та секретний ключ відправника, Verify(M, S, e, n) потребує повідомлення, підпис та відкритий ключ відправника. Кожна операція є лише піднесенням до степеня за модулем, тому їй потрібні тільки модуль і один показник (e або d). Прості числа p і q та φ(n) потрібні лише при генерації ключів, і їх передавання операціям було б зайвим та небезпечним, бо знання p і q дає змогу обчислити d. Операції шифрування та перевірки підпису використовують лише відкриті дані, тому їх може виконати будь-хто, а розшифрування та підпис потребують секретного d, яке знає лише власник.

4. Властивості протоколу.

Конфіденційність забезпечується шифруванням на ключі одержувача B. Відправник A обчислює $k_1 = k^{e_1} \bmod n_1$ та $S_1 = S^{e_1} \bmod n_1$ за відкритим ключем B, а розшифрувати їх може лише B за секретним d_1. Тому підслуховувач, який бачить

пару (k_1, S_1) по відкритому каналу, не може дізнатися ні k, ні підпис. Підтвердження справжності забезпечується підписом S = k^d mod n на секретному ключі A: створити такий підпис може лише власник d, а B перевіряє його відкритим ключем (e, n) абонента A. Вимога $n_1 \geq n$ потрібна, щоб підпис $S < n$ коректно вміщувався в модуль n_1 при шифруванні.

5. Перевірки одержувача.

Одержувач B розшифровує обидві частини своїм секретним ключем: $k = k_1^{d_1} \bmod n_1$ і $S = S_1^{d_1} \bmod n_1$. Потім він перевіряє, що підпис вірний, тобто $k = S^e \bmod n$ за відкритим ключем A. Лише якщо ця рівність виконується, k вважається коректним (не спотвореним при передачі) і справжнім (надісланим саме A). Якщо вона не виконується, значення відхиляється: у нашому прикладі при спотвореному S_1 програма виводить Спотворений S1 приймається одержувачем: False. Додатково перевіряється, що значення k лежить у допустимих межах ($0 < k < n$), а підпис S не виходить за межі $0 \leq S \leq n - 1$.